

Atelier guidé avec assistance de l'intelligence artificielle

But de l'atelier — concevoir deux représentations d'un même domaine dans PostgreSQL, les interroger en SQL, puis y accéder à partir de C# et de Python.

Objectifs d'apprentissage

- Créer et valider un schéma relationnel avec clés, contraintes et relations.
- Stocker et interroger des documents JSON avec le type JSONB et un index GIN.
- Comparer les compromis d'un modèle relationnel et d'un modèle documentaire.
- Se connecter à PostgreSQL de façon sécuritaire depuis C# et Python.
- Utiliser une IA générative comme aide, puis vérifier et expliquer le résultat obtenu.

Préparation du poste de travail

Installez les versions stables compatibles avec macOS et Apple Silicon (ARM64).

[PostgreSQL](#) 18.6

[pgAdmin 4](#) 9.17

[Visual Studio Code](#) — dernière version stable

[Python](#) 3.14.7

[.NET SDK](#) 10 LTS, installation ARM64

Extensions et bibliothèques requises : C# Dev Kit dans VS Code, Npgsql pour C#, et psycopg pour Python.

Outils d'IA autorisés

Vous pouvez utiliser un ou plusieurs assistants. Utilisez uniquement leur site officiel et ne leur transmettez jamais de mot de passe ni de donnée personnelle.

[ChatGPT](#)

[Gemini](#)

[Claude](#)

[Microsoft Copilot](#)

[Perplexity](#)

[Le Chat](#)

Mise en situation

Une petite bibliothèque souhaite gérer son catalogue, ses membres et ses emprunts. Elle veut comparer une solution relationnelle classique à une représentation documentaire JSONB. Les deux bases doivent contenir des données équivalentes afin que la comparaison soit significative.

Partie 1 — Base relationnelle SQL

1. **Créez la base `atelier_bibliotheque_sql`.** Utilisez un encodage UTF-8.
2. **Créez au minimum quatre tables :** auteur, livre, membre et emprunt. Ajoutez les clés primaires et étrangères appropriées.
3. **Ajoutez des contraintes pertinentes :** NOT NULL, UNIQUE, CHECK et valeurs par défaut. Justifiez au moins deux contraintes dans le README.
4. **Insérez un jeu de données cohérent :** au moins 5 auteurs, 10 livres, 5 membres et 8 emprunts, dont certains terminés et d'autres actifs.
5. **Écrire et tester les requêtes suivantes :**
 - une jointure affichant les emprunts avec le titre du livre et le nom du membre;
 - une agrégation indiquant le nombre d'emprunts par membre;
 - une requête filtrée et triée avec paramètres;
 - un INSERT, un UPDATE et un DELETE exécutés de façon sécuritaire;
 - une transaction comprenant COMMIT ou ROLLBACK.

Partie 2 — Modèle documentaire avec JSONB

1. **Créez la base `atelier_bibliotheque_jsonb`.** Créez une table document comportant au minimum id, type_document, donnees JSONB et cree_le.
2. **Représentez les mêmes données sous forme de documents JSON.** Chaque document doit comporter une structure réaliste, incluant au moins un tableau ou un objet imbriqué.
3. **Validez les documents.** Ajoutez au moins une contrainte vérifiant une propriété JSON essentielle et créez un index GIN sur la colonne JSONB.
4. **Écrire et tester les requêtes JSONB suivantes :**
 - recherche par propriété avec les opérateurs `->`, `->>` ou `#>>`;
 - recherche par contenance avec `@>`;
 - recherche dans un tableau JSON;
 - mise à jour ciblée d'une propriété avec `jsonb_set`;
 - requête démontrant l'utilisation de l'index, accompagnée d'un EXPLAIN.

Comparaison obligatoire — dans le README, expliquez en 150 à 250 mots quel modèle vous choisiriez pour ce système et pourquoi. Appuyez-vous sur la structure, l'intégrité, la flexibilité et les requêtes réalisées.

Partie 3 — Application console en C#

Créez une application console .NET utilisant le pilote **Npgsql**. Elle doit se connecter aux deux bases et proposer un menu simple.

- afficher la liste des livres de la base relationnelle;
- rechercher un livre ou un auteur à partir d'une valeur saisie par l'utilisateur;
- afficher une information équivalente provenant de la base JSONB;
- ajouter ou modifier une donnée avec une requête paramétrée;
- intercepter les erreurs et fermer proprement les connexions.

Partie 4 — Application console en Python

Créez une application Python utilisant **psycopg**. Elle doit offrir les mêmes quatre opérations principales que l'application C#, afin de faciliter la comparaison des deux langages.

- isoler la connexion et les requêtes dans des fonctions;
- utiliser des paramètres plutôt que de concaténer les saisies dans le SQL;
- lire les paramètres de connexion dans des variables d'environnement ou un fichier .env exclu de la remise;
- afficher des messages clairs lorsque la connexion ou une requête échoue.

Exigences communes aux deux programmes

- Aucun mot de passe ne doit apparaître dans le code source, le README, les captures ou l'historique de l'IA.
- Toutes les valeurs provenant de l'utilisateur doivent être transmises au moyen de requêtes paramétrées.
- Le projet doit pouvoir être exécuté en suivant uniquement les instructions du README.
- Vous devez être capable d'expliquer chaque requête et chaque portion de code remises.

Validation avant la remise

- ☐ Les deux scripts de création s'exécutent à partir d'une base vide.
- ☐ Les données SQL et JSONB représentent le même domaine.
- ☐ Toutes les requêtes demandées produisent un résultat vérifiable.
- ☐ Les applications C# et Python se lancent sans modification du code.
- ☐ Les informations sensibles sont absentes de tous les fichiers remis.

Remise

Remettez une archive nommée **Atelier1.zip** avec la structure suivante :

- sql/01_creation_relationnelle.sql
- sql/02_donnees_relationnelles.sql
- sql/03_requetes_relationnelles.sql
- sql/04_creation_jsonb.sql
- sql/05_donnees_et_requetes_jsonb.sql
- csharp/ — projet exécutable sans dossiers bin et obj
- python/ — code source et fichier requirements.txt
- README.md — installation, exécution, comparaison des modèles et justification des choix
- IA.md — assistants utilisés, principaux prompts, corrections apportées et vérifications effectuées

Usage responsable de l'intelligence artificielle

L'IA est autorisée comme partenaire d'apprentissage, mais elle ne remplace ni la validation ni la compréhension. Toute proposition générée doit être testée dans votre environnement.

Vous devez

- consigner dans IA.md les outils utilisés et les principaux prompts;
- signaler les erreurs repérées dans les réponses de l'IA et expliquer vos corrections;
- vérifier les noms de fonctions, les versions, la syntaxe SQL et le comportement réel du code;
- citer toute source externe reprise substantiellement.

Vous ne devez pas

- remettre du code que vous êtes incapable d'expliquer ou de modifier;
- transmettre des identifiants, mots de passe ou données personnelles à un assistant;
- présenter comme vôtre un résultat non vérifié ou masquer l'utilisation de l'IA.

Démonstration — l'enseignant peut demander une courte démonstration et une modification en direct afin de vérifier la compréhension du schéma, des requêtes et du code.

Critères de réussite

Dimension	Indicateur
Modélisation	clés, relations, contraintes et structure JSONB cohérentes
Requêtes	exactes, variées, paramétrées et testées
Applications	connexion fiable, fonctions demandées et gestion des erreurs
Comparaison	analyse claire des avantages et limites des deux modèles
Démarche	README reproductible, traces d'IA transparentes et code compris